

ARDUINO DE PLACA DE PRESION

****Componentes:****

- * Arduino Uno: Placa de microcontrolador que controla todo el sistema.
- * LED Rojo: Diodo emisor de luz que se ilumina para indicar una salida.
- * Resistencia (220Ω): Limitador de corriente para el LED, protegiéndolo de daños por sobrecorriente.
- * Potenciómetro: Dispositivo que permite variar la resistencia y que se usa para controlar la intensidad del LED.
- * Cables de conexión: Conectan los componentes al circuito.
- * Protoboard (Placa de pruebas): Superficie para conectar los componentes fácilmente.

****Descripción:****

El circuito es un sistema básico para controlar la luminosidad de un LED rojo utilizando un potenciómetro. El potenciómetro se conecta a una entrada analógica del Arduino. El valor leído de esta entrada (de 0 a 1023) refleja la posición del potenciómetro, que luego se utiliza para controlar la salida PWM (modulación de ancho de pulso) del pin digital del Arduino conectado al LED a través de la resistencia limitadora de corriente. Un valor más alto del potenciómetro resultará en una mayor luminosidad del LED.

****Código:****

```arduino

const int ledPin = 9; // Pin digital para el LED

const int potPin = A0; // Pin analógico para el potenciómetro

void setup() {

pinMode(ledPin, OUTPUT); // Configura el pin del LED como salida

Serial.begin(9600); // Inicia la comunicación serial (opcional, para monitoreo)

}

void loop() {

int potValue = analogRead(potPin); // Lee el valor del potenciómetro

// Mapea el valor del potenciómetro a un rango de 0 a 255 para el PWM

int brightness = map(potValue, 0, 1023, 0, 255);

analogWrite(ledPin, brightness); // Configura la luminosidad del LED

// Salida serial (opcional, para monitoreo)

Serial.print("Valor del potenciómetro: ");

Serial.print(potValue);

Serial.print(" Brillo del LED: ");

Serial.println(brightness);

delay(100); // Retardo pequeño para estabilidad

}

...

Este código lee el valor del potenciómetro a través del pin analógico A0, mapea este valor a un rango de 0 a 255 (que son los niveles de PWM para el Arduino Uno) y luego usa la función `'analogWrite()'` para controlar la luminosidad del LED conectado al pin digital 9. La comunicación serial es opcional, pero permite observar los valores del potenciómetro y el brillo del LED en el monitor serial del IDE de Arduino. Recuerda que necesitas tener la librería "ArduinoJson" para usar el código si optas por la parte de la comunicación serial.

Recuerda conectar el circuito correctamente según la imagen. La resistencia en serie con el LED es esencial para protegerlo. El ánodo (pata más larga) del LED debe conectarse al pin digital 9 del Arduino a través de la resistencia, mientras que el cátodo (pata más corta) se conecta a GND. La conexión del potenciómetro va de 5V a A0, y la otra patita a GND. En el esquema se muestra el diagrama de conexión.

## ARDUINO DE SEMAFORO

### **\*\*Componentes:\*\***

\* **\*\*Arduino Uno:\*\*** Una placa de microcontrolador que forma el cerebro del circuito.

\* **\*\*LEDs (Rojo, Amarillo, Verde):\*\*** Tres diodos emisores de luz de diferentes colores que se utilizan como salida visual.

\* **\*\*Resistencias (220Ω):\*\*** Tres resistencias que limitan la corriente que fluye hacia los LEDs, protegiéndolos de daños.

\* **\*\*Placa de pruebas (protoboard):\*\*** Una placa utilizada para conectar fácilmente los componentes y crear el circuito.

\* **\*\*Cables de conexión (jumpers):\*\*** Cables utilizados para conectar los componentes en la placa de pruebas.

### **\*\*Descripción:\*\***

El circuito muestra un simple programa que controla tres LEDs (rojo, amarillo y verde) conectados a un Arduino Uno. Cada LED está conectado a un pin digital diferente del Arduino a través de una resistencia de 220Ω. El código Arduino controla el estado de cada LED individualmente, encendiéndolos y apagándolos en una secuencia o patrón determinado. Esto permite crear una variedad de efectos de iluminación. Las resistencias son cruciales para limitar la corriente que fluye a los LEDs y evitar que se quemen.

**\*\*Código:\*\***

```
```arduino
```

```
// Definir los pines a los que están conectados los LEDs
```

```
const int ledRojo = 2; // Pin digital 2 para el LED rojo
```

```
const int ledAmarillo = 3; // Pin digital 3 para el LED amarillo
```

```
const int ledVerde = 4; // Pin digital 4 para el LED verde
```

```
void setup() {
```

```
    // Configurar los pines como salidas
```

```
    pinMode(ledRojo, OUTPUT);
```

```
    pinMode(ledAmarillo, OUTPUT);
```

```
    pinMode(ledVerde, OUTPUT);
```

```
}
```

```
void loop() {
```

```
    // Encender el LED rojo
```

```
    digitalWrite(ledRojo, HIGH);
```

```
    delay(1000); // Esperar 1 segundo
```

```
    digitalWrite(ledRojo, LOW); // Apagar el LED rojo
```

```
    // Encender el LED amarillo
```

```
    digitalWrite(ledAmarillo, HIGH);
```

```
    delay(1000); // Esperar 1 segundo
```

```
    digitalWrite(ledAmarillo, LOW); // Apagar el LED amarillo
```

```
    // Encender el LED verde
```

```
    digitalWrite(ledVerde, HIGH);
```

```
    delay(1000); // Esperar 1 segundo
    digitalWrite(ledVerde, LOW); // Apagar el LED verde

}
...
```

Este código primero define los pines digitales a los que están conectados los LEDs. Luego, en la función `setup()`, configura estos pines como salidas. En la función `loop()`, que se ejecuta repetidamente, el código enciende cada LED individualmente, espera un segundo y luego lo apaga. Puedes modificar este código para crear secuencias de luces más complejas o patrones personalizados. Recuerda ajustar los números de pines si tus LEDs están conectados a diferentes pines en tu placa Arduino.

ARDUINOS DE SENSORES CLIMATICOS

****Componentes:****

- * Placa de desarrollo Arduino Uno
- * Placa de pruebas
- * Sensor ultrasónico HC-SR04
- * Sensor de humo MQ-2
- * Motor de CC
- * Pantalla LCD 16x2
- * Resistencia de 220 ohmios
- * Resistencia de 10k ohmios
- * LED
- * Cables de conexión

****Descripción:****

Este circuito utiliza un Arduino Uno para monitorizar simultáneamente un sensor de humo y un sensor ultrasónico. El sensor de humo (MQ-2) mide los niveles de humo en el ambiente. Si detecta humo por encima de un umbral predefinido, activa un LED y un motor de CC para indicar una emergencia. El sensor ultrasónico (HC-SR04) mide la distancia a objetos cercanos. Si un objeto se acerca demasiado (por debajo de un umbral predefinido), el circuito mostrará la distancia en la pantalla LCD. La pantalla LCD también muestra las lecturas del sensor de humo.

****Código:****

```
```.cpp
#include <LiquidCrystal.h>

// Pines para el sensor ultrasónico
const int trigPin = 9;
const int echoPin = 10;

// Pines para el sensor de humo
const int humoPin = A0;
const int ledPin = 13;
const int motorPin = 8;

// Pines para la pantalla LCD
const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

void setup() {
 pinMode(trigPin, OUTPUT);
 pinMode(echoPin, INPUT);
 pinMode(ledPin, OUTPUT);
 pinMode(motorPin, OUTPUT);
 Serial.begin(9600);
 lcd.begin(16, 2);
 lcd.print("Sistema de ");
 lcd.setCursor(0, 1);
 lcd.print("Deteccion");
}
```



```
void loop() {
 long duracion, distancia;
 int nivelHumo;

 // Medición del sensor ultrasónico
 digitalWrite(trigPin, LOW);
 delayMicroseconds(2);
 digitalWrite(trigPin, HIGH);
 delayMicroseconds(10);
 digitalWrite(trigPin, LOW);
 duracion = pulseIn(echoPin, HIGH);
 distancia = duracion * 0.034 / 2;

 // Lectura del sensor de humo
 nivelHumo = analogRead(humoPin);

 // Mostrar datos en la pantalla LCD
 lcd.clear();
 lcd.print("Distancia: ");
 lcd.print(distancia);
 lcd.print(" cm");
 lcd.setCursor(0, 1);
 lcd.print("Humo: ");
 lcd.print(nivelHumo);

 // Respuesta al humo
```

```

 if (nivelHumo > 500) { //Ajustar el umbral según la
calibración del sensor
 digitalWrite(ledPin, HIGH);
 digitalWrite(motorPin, HIGH);
 Serial.println("Alarma de humo!");
 } else {
 digitalWrite(ledPin, LOW);
 digitalWrite(motorPin, LOW);
 }

 // Respuesta a la proximidad
 if (distancia < 20) { //Ajustar el umbral según las
necesidades
 Serial.println("Objeto cercano detectado!");
 }
 delay(100);
}
...

```

Recuerda que necesitas calibrar los umbrales de los sensores de humo y distancia según tu entorno específico. Este código proporciona una base funcional. Podrás modificarlo para incluir otras funcionalidades o mejorar la precisión. Además, asegúrate de conectar los componentes a los pines correctos del Arduino según el diagrama esquemático.

## ARDUINO DE RECONOCIMIENTO DE VOZ

### Componentes:

- \* Placa Arduino (parece una placa Genuino Uno o similar)
- \* Placa de pruebas
- \* 3 LEDs (rojo, amarillo y verde)
- \* Resistencias (para limitar la corriente a los LEDs)
- \* Buzzer piezoeléctrico
- \* Sensor ultrasónico (HC-SR04 o similar)
- \* Cables de puente

### \*\*Descripción:\*\*

Este circuito utiliza un sensor ultrasónico para detectar la distancia de un objeto. Dependiendo de la distancia detectada, el circuito activa diferentes LEDs y el zumbador.

\* **\*\*Sensor ultrasónico:\*\*** El sensor ultrasónico envía ondas de ultrasonido y mide el tiempo que tardan en regresar después de rebotar en un objeto. Este tiempo se usa para calcular la distancia al objeto.

\* **\*\*Placa Arduino:\*\*** La placa Arduino lee la distancia medida por el sensor ultrasónico.

\* **\*\*Programación:\*\*** El código Arduino luego procesa la distancia y controla los LEDs y el zumbador según la distancia detectada. Por ejemplo:

\* Distancia corta (objeto cercano): el LED rojo se enciende y el zumbador suena.

\* Distancia media (objeto a distancia moderada): el LED amarillo se enciende.

\* Distancia larga (objeto lejano): el LED verde se enciende.

### **\*\*Código:\*\***

Este código asume que el sensor ultrasónico está conectado a los pines digitales 7 y 8 de Arduino, y los LEDs rojo, amarillo y verde a los pines 12, 11 y 10, respectivamente. El buzzer está conectado al pin 9. Ajusta estos pines según tu esquema real de conexión.

```
``cpp
```

```
const int trigPin = 7;
const int echoPin = 8;
const int redLed = 12;
const int yellowLed = 11;
const int greenLed = 10;
const int buzzerPin = 9;
```

```
long duration;
```

```
int distance;
```

```
void setup() {
 pinMode(trigPin, OUTPUT);
 pinMode(echoPin, INPUT);
 pinMode(redLed, OUTPUT);
 pinMode(yellowLed, OUTPUT);
 pinMode(greenLed, OUTPUT);
 pinMode(buzzerPin, OUTPUT);
 Serial.begin(9600);
}
```

```

void loop() {
 // Clears the trigPin
 digitalWrite(trigPin, LOW);
 delayMicroseconds(2);

 // Sets the trigPin HIGH (ACTIVE) for 10 microseconds
 digitalWrite(trigPin, HIGH);
 delayMicroseconds(10);
 digitalWrite(trigPin, LOW);

 // Reads the echoPin, returns the sound wave travel time in
 microseconds
 duration = pulseIn(echoPin, HIGH);

 // Calculating the distance
 distance = duration * 0.034 / 2;

 // Prints the distance on the Serial Monitor
 Serial.print("Distance: ");
 Serial.println(distance);

 // Controla los LEDs y el zumbador
 if (distance <= 10) {
 digitalWrite(redLed, HIGH);
 tone(buzzerPin, 1000); // Frecuencia del zumbador
 } else if (distance <= 20) {
 digitalWrite(redLed, LOW);
 }
}

```

```
 digitalWrite(yellowLed, HIGH);
 noTone(buzzerPin);
} else {
 digitalWrite(yellowLed, LOW);
 digitalWrite(greenLed, HIGH);
 noTone(buzzerPin);
}
delay(100); // Espera antes de la siguiente lectura
digitalWrite(redLed, LOW);
digitalWrite(yellowLed, LOW);
digitalWrite(greenLed, LOW);
}
...
```

Recuerda ajustar el código a tu conexión específica y probarlo para verificar el funcionamiento correcto. Los umbrales de distancia (10 y 20 cm en este ejemplo) pueden ser ajustados según tus necesidades.